CS458 Software Verification and Validation

Project 1 Proposal

Maksat Abrayev – 22201182

Hüseyin Utku Yüksel – 22103511

Farabi Sina Sarı – 22102084

Furkan Özek – 22103680

1. Introduction..... 3
2. System Overview..... 3
2.1 Technology Stack..... 3
2.2 Project Structure..... 4
3. Authentication System..... 4
3.1 Supported Login Methods..... 4
3.2 API Endpoints..... 4
4. Risk Assessment Engine..... 5
4.1 Risk Factors..... 5
4.2 Risk Levels..... 5
5. Account State Machine..... 6
5.1 State Transitions..... 6
Table:..... 6
Diagram:..... 6
6. OAuth2 Authentication Flow..... 7
Flow Summary: Login by 3rd Party Applications..... 7
Flow Summary: Login by Ares Native..... 7
7. Self-Healing Selenium Framework..... 8
7.1 Healing Strategy..... 8
8. Automated Test Scenarios..... 9
8.0 Scenario Coverage Summary..... 9
8.1 Dynamic ID Recovery..... 11
8.2 Multimodal Failure..... 12
8.3 Cross-Browser / CSS Resilience..... 13
8.4 Social Auth Handshake..... 14
8.5 Brute-force..... 15
9. LLM Integration..... 15
9.1 LLM Prompt Templates (Available in the codebase)..... 15
9.1.1 Fraud Analysis Prompt..... 16
9.1.2 Selector Repair Prompt..... 16
9.1.3 Interaction Unblock Prompt..... 16
9.2 Test-Case-to-Prompt Mapping (with File Locations)..... 17
9.4 Current Healing Behavior (Post-Refactor)..... 18
10. Registered Users..... 18
11. Environment Variables..... 19
12. Limitations..... 20

1. Introduction

ARES Auth (AI-Driven Resilient & Evolutionary Authentication System) is an adaptive authentication platform integrated with a self-healing Selenium testing framework.

The system combines:

- Context-aware credential authentication
- OAuth2 social login (Google, Facebook)
- Risk-based login scoring
- LLM-assisted fraud evaluation (Gemini 2.0 Flash)
- AI-powered DOM selector repair for resilient automated testing

The objective is to demonstrate adaptive security behavior and resilient automated verification under UI evolution.

2. System Overview

2.1 Technology Stack

Layer	Technology
Frontend	Next.js 16.1.6 (App Router), React 19.2.3
Authentication	next-auth v4
AI / Fraud Analysis	Gemini 2.0 Flash (@google/generative-ai)
Password Hashing	bcryptjs
Backend State	In-memory user store
Test Suite	Python + pytest + Selenium + requests

2.2 Project Structure

Folder	Responsibility
`app/`	UI pages and API routes
`lib/`	Users, Risk Engine, LLM client
`tests/`	Self-healing Selenium framework + scenario suites

3. Authentication System

3.1 Supported Login Methods

- Email/Phone + Password
- Google OAuth
- Facebook OAuth

Credential login is handled via:

POST /api/login

On successful authentication, users are redirected to:

/dashboard

On a successful but suspicious entry, credentials and entry source info are sent to LLM for further risk analysis. If LLM also finds suspicious, the account is locked until it is re-authanticated by 2FA, or the connected app is authorized again.

3.2 API Endpoints

Method	Endpoint	Purpose
POST	/api/login	Credential login + risk scoring
POST	/api/risk-assessment	Risk engine evaluation
POST	/api/heal	Selector repair using LLM
GET/POST	/api/reset-account	Account reset
GET/POST	/api/auth/[...nextauth]	OAuth handling

4. Risk Assessment Engine

The risk engine is implemented in:

/lib/riskEngine.js

Risk scoring evaluates contextual login properties and produces a score between 0 and 100.

4.1 Risk Factors

Factor	Score Contribution
Unknown IP	+25
Failed attempts	+8 each (max +40)
IP attempt pressure (>5 attempts)	+(attempts−5)*5 (max +20)
Unusual hour (02:00–05:59)	+10
Challenged status	+10
Locked status	+30
Suspended status	+50

Score is capped at 100.

4.2 Risk Levels

Score Range	Level
< 30	Low
≥ 30	Medium
≥ 50	High
≥ 70	Critical

When the score is **≥ 50**, unknown IP is detected or failed attempts >= 10 then LLM-based fraud analysis is triggered. If LLM fails, a rule-based fallback decision is applied.

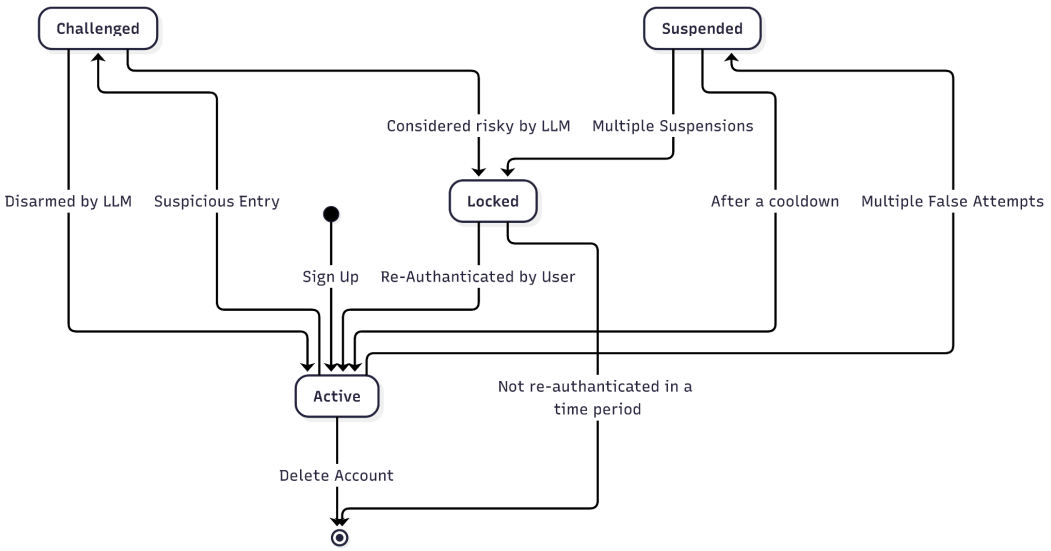
5. Account State Machine

5.1 State Transitions

Table:

Source State	Condition	Resulting State
Initial State	Sign Up	Active
Active	Delete Account	Final State
Active	Suspicious Entry	Challenged
Active	Multiple False Attempts	Suspended
Challenged	Disarmed by LLM	Active
Challenged	Considered risky by LLM	Locked
Suspended	Multiple Suspensions	Locked
Suspended	After a cooldown	Active
Locked	Re-Authenticated by User	Active
Locked	Not re-authenticated in a time period	Final State

Diagram:



User accounts are stored in memory and maintain:

- failedAttempts
- known IP addresses
- account status

6. OAuth2 Authentication Flow

Flow Summary: Login by 3rd Party Applications

1. User clicks Google or Facebook login.
2. Redirect to provider.
3. Provider authenticates user.
4. next-auth handles token exchange.
5. User session is created.
6. Redirect to /dashboard.

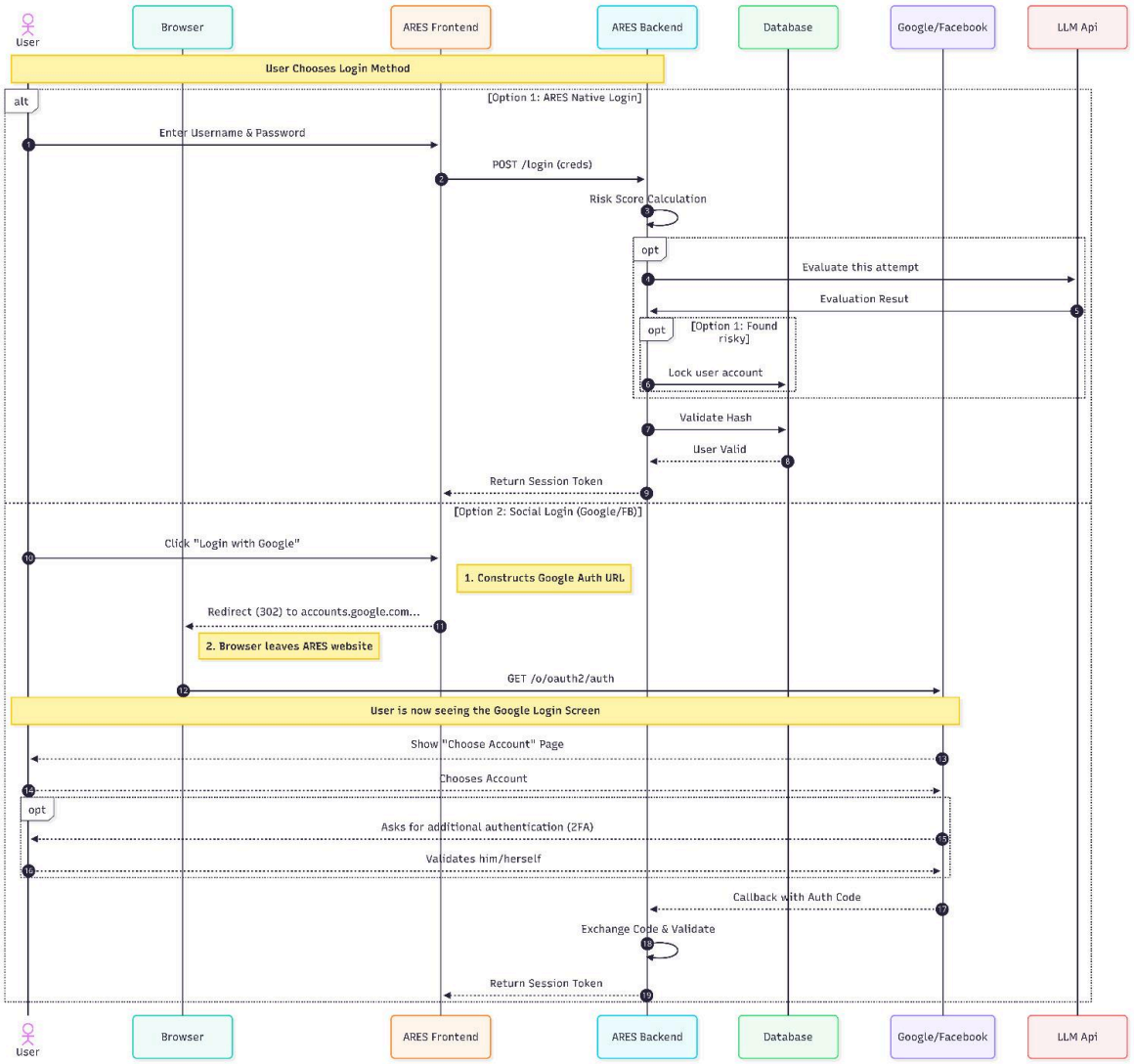
Risk assessment is integrated in credential login flow and impacts account state.

OAuth is implemented using next-auth v4.

Flow Summary: Login by Ares Native

7. User posts username and password to ARES server.
8. Server evaluates and validates credentials.
9. Token exchange.
10. User session is created.

Risk assessment is integrated in credential login flow and impacts account state.



7. Self-Healing Selenium Framework

The test framework is implemented in Python using Selenium and pytest.

Healing logic is encapsulated in a SelfHealingDriver.

7.1 Healing Strategy

When an element lookup fails:

1. Attempt normal Selenium lookup.
2. Attempt heuristic repair using stored metadata.

3. Send failed selector + DOM to /api/heal.
4. If needed, call Gemini directly (fallback).
5. Retry selector after repair.
6. Log healing event to JSON report.

Strict JSON selector output is required for repair responses.

8. Automated Test Scenarios

8.0 Scenario Coverage Summary

#	Scenario	What is intentionally broken/simulated	Expected system behavior	Exact test locations
1	Dynamic ID Recovery	Element IDs are changed at runtime (login-btn, identifier, tab-email, google-login-btn)	Self-healing should recover new selectors and continue test flow	tests/test_dynamic_id.py:19, tests/test_dynamic_id.py:60, tests/test_dynamic_id.py:99
2	Multimodal Failure	Click target exists but is blocked by popup/overlay; close-button ID also mutated in one case	Framework should resolve blocker (LLM/unblock path) and complete intended click	tests/test_multimodal_failure.py:20, tests/test_multimodal_failure.py:70, tests/test_multimodal_failure.py:115

3	Cross-Browser/CSS Resilience	CSS breakage, class removal, DOM repositioning, visual variable corruption	Core elements remain discoverable; healing triggers where selectors become invalid	tests/test_cross_browser.py:24, tests/test_cross_browser.py:67, tests/test_cross_browser.py:112, tests/test_cross_browser.py:138
4	Social Auth Handshake	OAuth redirect flow checks + social button ID mutation	Redirect URL patterns remain valid; self-healing recovers mutated OAuth button IDs	tests/test_social_auth.py:17, tests/test_social_auth.py:63, tests/test_social_auth.py:92, tests/test_social_auth.py:128
5	Brute-force / Rate-Limiting Simulation	Repeated failed login attempts via UI and API; risk escalation path	Account state escalates (active -> challenged -> locked -> suspended), risk score increases, LLM fraud path may trigger at high risk	tests/test_rate_limiting.py:19, tests/test_rate_limiting.py:84, tests/test_rate_limiting.py:154

8.1 Dynamic ID Recovery

Test	Mutation Injected	Healing Input (from test description)	Validation
test_login_button_id_change	login-btn -> submit-auth-btn	The main login/sign-in button that submits the form	Healed element found, text still Sign In, healing log recorded
test_input_field_id_change	identifier -> user-email-input	Email/phone input field for login	Healed input found and remains usable
test_multiple_id_changes	login-btn -> new-login-btn, tab-email -> new-tab-email, google-login-btn -> new-google-btn	Main login button, Email tab button, Login with Google button	At least 3 healing events expected

Implementation locations:

- tests/test_dynamic_id.py:19
- tests/test_dynamic_id.py:60
- tests/test_dynamic_id.py:99

8.2 Multimodal Failure

Test	Failure Type	Healing/Recovery Path	Validation
test_popup_obscures_google_login	Click intercepted by overlay popup	click_element_resilient(...) invokes unblock flow	Popup dismissed, click succeeds, healing logged
test_popup_with_changed_close_button	Popup close button ID changed (close-popup-btn -> accept-cookies-btn)	Selector repair flow on old close-button ID	Healed close button found and popup closed
test_multiple_overlapping_popups	Multiple blockers stacked	Sequential interaction recovery	Banner dismissed, popup closed, login button accessible

Implementation locations:

- tests/test_multimodal_failure.py:20
- tests/test_multimodal_failure.py:70
- tests/test_multimodal_failure.py:115

8.3 Cross-Browser / CSS Resilience

Test	Disturbance	Self-healing role	Validation
test_css_display_none_breakage	Login button hidden with display:none	Primarily recovery-by-restoration (not LLM-centric)	Button visible and clickable after fix
test_css_class_removal	Classes removed, then ID removed	Strong healing trigger via broken CSS selector lookup	Element still found or healing log produced
test_element_repositioning	Google button moved in DOM tree	Usually no repair needed if ID preserved	Button still present/displayed
test_font_and_color_breakage	CSS variables corrupted	Functional resilience check	Inputs/buttons still found

Implementation locations:

- tests/test_cross_browser.py:24
- tests/test_cross_browser.py:67
- tests/test_cross_browser.py:112
- tests/test_cross_browser.py:138

8.4 Social Auth Handshake

Test	Focus	LLM/Self-healing relevance	Validation
test_google_redirect_flow	Google OAuth redirect behavior	No forced selector breakage	URL indicates OAuth/NextAuth/signin path
test_facebook_redirect_flow	FB OAuth redirect beh.	No forced selector breakage	URL indicates OAuth/NextAuth/signin path
test_social_auth_button_presence_after_id_change	Social button ID mut.	Direct selector repair usage	Healed Google + Facebook buttons discovered
test_oauth_callback_url_structure	Provider metadata endpoint	No selector healing	Providers endpoint includes configured providers

Implementation locations:

- tests/test_social_auth.py:17
- tests/test_social_auth.py:63
- tests/test_social_auth.py:92
- tests/test_social_auth.py:128

8.5 Brute-force

Test	Channel	Purpose	Validation
test_brute_force_attempt	UI	Repeated wrong-password attempts and account status progression	Escalation observed in page status/messages
test_brute_force_via_api	/api/login	Faster brute-force simulation, risk progression	Risk does not decrease; restricted state reached
test_risk_score_escalation	/api/risk-assessment	Compare known IP vs unknown IP risk scoring	Unknown IP risk >= baseline; factors reported

Implementation locations:

- tests/test_rate_limiting.py:19
- tests/test_rate_limiting.py:84
- tests/test_rate_limiting.py:154

9. LLM Integration

9.1 LLM Prompt Templates (Available in the codebase)

Prompt source file:

- lib/llmClient.js:44 (analyzeFraud)
- lib/llmClient.js:119 (repairSelector)
- lib/llmClient.js:182 (resolveInteractionBlocker)

9.1.1 Fraud Analysis Prompt

Used by analyzeFraud(...)

Inputs: userEmail, ip, riskScore, riskLevel, failedAttempts, accountStatus, factors, timestamp

Output contract: JSON with isFraudulent, confidence, reasoning, recommendedAction

Runtime call sites:

- app/api/login/route.js:69
- app/api/risk-assessment/route.js:27
- lib/oauthSecurity.js:29

9.1.2 Selector Repair Prompt

Used by repairSelector(...)

Inputs: oldSelector, elementDescription, domSnippet

Output contract: JSON with newSelector, selectorType, confidence, reasoning

Runtime/API path:

- app/api/heal/route.js:30 -> lib/llmClient.js:106
- Test caller path: tests/self_healing.py:161 -> tests/llm_repair.py:33 -> /api/heal

9.1.3 Interaction Unblock Prompt

Used by resolveInteractionBlocker(...)

Inputs: targetSelector, actionDescription, domSnippet

Output contract: JSON with blockerSelector, selectorType, action, confidence, reasoning

Runtime/API path:

- app/api/heal/route.js:18 -> lib/llmClient.js:168
- Test caller path: tests/self_healing.py:304 -> tests/llm_repair.py:97 -> /api/heal

9.2 Test-Case-to-Prompt Mapping (with File Locations)

Test case	Prompt template used	Test-provided prompt text (injected variable)	Exact location
Dynamic ID tests	Selector Repair	The main login/sign-in button that submits the form, Email/phone input field for login, Main login button, Email tab button, Login with Google button	tests/test_dynamic_id.py:49, tests/test_dynamic_id.py:89, tests/test_dynamic_id.py:123, tests/test_dynamic_id.py:129, tests/test_dynamic_id.py:135
Multimodal popup blocked click	Interaction Unblock	Click Google login button while popup may block the interaction	tests/test_multimodal_failure.py:57
Multimodal changed close button	Selector Repair	Button to accept cookies and close the popup overlay	tests/test_multimodal_failure.py:98
Cross-browser class/id breakage	Selector Repair (when lookup fails)	Primary login submit button with gradient styling	tests/test_cross_browser.py:107

Social auth with changed IDs	Selector Repair	Google OAuth login button with Google logo, Facebook OAuth login button with Facebook logo	tests/test_social_auth.py:113, tests/test_social_auth.py:120
Brute-force/ risk tests	Fraud Analysis (conditionally, score threshold)	Risk context is constructed in API route, not hardcoded in test string	tests/test_rate_limiting.py:84, app/api/login/route.js:69

9.4 Current Healing Behavior (Post-Refactor)

Test-side LLM client is API-only, single-call, fail-fast (no test-side Gemini fallback loops).
File: tests/llm_repair.py:18-160

API returns semantic success/failure based on usable selector presence.
File: app/api/heal/route.js:37-51

LLM errors are classified (rate_limited vs llm_error).
File: lib/llmClient.js:12-18

Gemini model is configurable via GEMINI_MODEL (default gemini-2.5-flash).
File: lib/llmClient.js:5

10. Registered Users

Source file:

- lib/data/userAccounts.json

id	email	phone	initialPassword	name	status	failedAttempts	knownIPs
1	testuser@ares.com	+905551234567	Test@1234	Test User	active	0	["127.0.0.1","::1"]
2	admin@ares.com	+905559876543	Admin@5678	Admin User	active	0	["127.0.0.1","::1"]
3	h.utku64@gmail.com	+905550001111	GithubTest@123	HuseyinUtkuYuksel	active	0	["127.0.0.1","::1"]
4	h.utku45@gmail.com	+905550002222	GoogleTest@123	Utku Yüksel	active	0	["127.0.0.1","::1"]
5	seniorprojecteway@gmail.com	+905550003333	Suspended@123	Suspended Hacker	suspended	5	[]

11. Environment Variables

Variable	Purpose
NEXTAUTH_URL	Base application URL
NEXTAUTH_SECRET	Session encryption
GOOGLE_CLIENT_ID	Google OAuth

GOOGLE_CLIENT_SECRET	Google OAuth
FACEBOOK_CLIENT_ID	Facebook OAuth
FACEBOOK_CLIENT_SECRET	Facebook OAuth
GEMINI_API_KEY	Gemini access
GITHUB_CLIENT_ID	Github OAuth
GITHUB_CLIENT_SECRET	Github Access

12. Limitations

- No persistent database (state resets on restart).
- Gemini rate limits may affect healing tests.
- OAuth flows depend on correct provider configuration.
- In-memory user model limits scalability.